PHINMA-COC

Carmen, Cagayan de Oro City

DATABASE SCHEMA SPECIFICATION

`cherry_cosmetics`

Desktop-Based Ordering System for Cherry Cosmetic Shop

Course: ITE-298 Information Management (Database Systems 1 and 2)

Submitted by:

Team Leader

John Carl S. Vanzuela

Members

Cherrygel P. Antivo
Chrysler John M. Roble
Elijah Shiene F. Canillo
Jevy G. Sultan

1. Database Overview

This document describes the database structure of the Desktop-Based Ordering System for Cherry Cosmetics Shop. The database stores customer accounts, product categories, product information, customer orders, order items, and payment records.

Database Name	cherry_cosmetics
RDBMS Engine	MySQL
Default Character Set	utf8mb4
Collation	Java Swing desktop application
Normalization Standard	Designed toward Third Normal Form (3NF)
Total Tables / Views	6 tables, 0 reporting views

2. Entity Relationship Diagram

Figure 1 shows the logical ERD in Crow's Foot notation. ORDER_ITEMS is the associative (junction) entity that resolves the Many-to-Many relationship between ORDERS and PRODUCTS into two One-to-Many relationships. USERS, CATEGORIES, and PRODUCTS store account, category, and product information, while PAYMENTS stores payment records associated with orders.

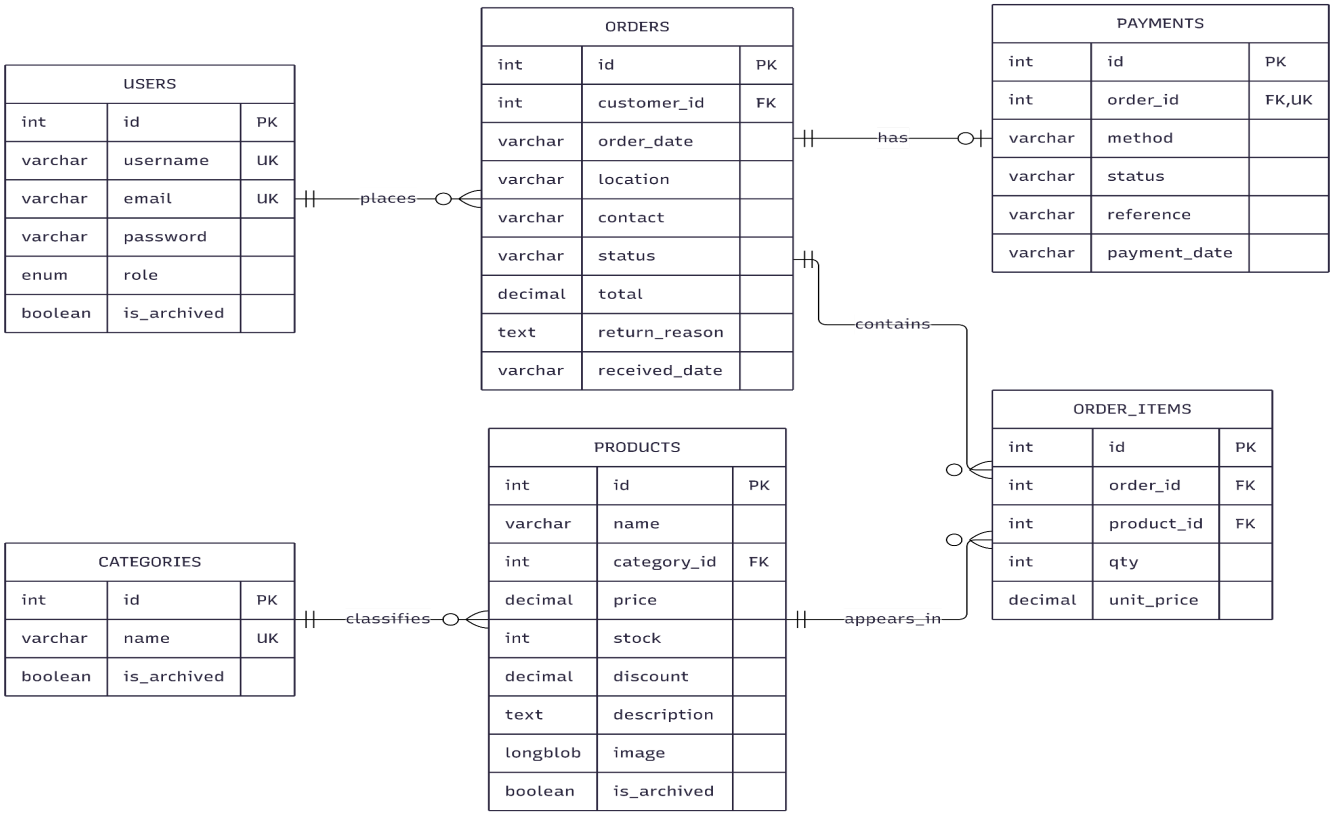


Figure 1. Crow's Foot ERD of cherry_cosmetics.db (PK, UK, FK and cardinalities).

Relationship Summary

Relationship	Type	Cardinality
USERS → ORDERS	1:M (Optional)	One customer can place zero or many orders. Each order belongs to one customer.
CATEGORIES → PRODUCTS	1:M (Optional)	One category can contain zero or many products. Each product belongs to one category.
ORDERS → ORDER_ITEMS	1:M (Optional)	One order can contain one or more order items. Each order item belongs to one order.
PRODUCTS → ORDER_ITEMS	1:M (Optional)	One product can appear in zero or many order items. Each order item refers to one product.
ORDERS ↔ PRODUCTS	M:N (resolved)	ORDER_ITEMS connects orders and products through order_id and product_id.
ORDERS → PAYMENTS	1:0..1	One order can have zero or one payment record. Each payment belongs to one order.

3. Table Dictionary & Data Specifications

1. USERS

Manages system user accounts, login credentials, and role-based access for administrators and customers.

Column	Data Type	Null	Key	Default	Description
id	INT	NO	PK, AUTO_INCREMENT	—	Unique account identifier
username	VARCHAR(100)	NO	UNIQUE	—	Account login username
email	VARCHAR(190)	NO	UNIQUE	—	Account email address
password	VARCHAR(255)	NO	—	—	Stored account password
role	ENUM('Admin','Customer')	NO	CHECK	—	Account role: Admin or Customer
is_archived	TINYINT(1)	NO	—	0	Indicates whether the account is archived

2. CATEGORIES

Stores unique product category names and the archive status of each category.

Column	Data Type	Null	Key	Default	Description
id	INT	NO	PK, AUTO_INCREMENT	AUTO_INCREMENT	Unique category identifier
name	VARCHAR(100)	NO	UNIQUE	None	Product category name
is_archived	TINYINT(1)	NO	—	—	Indicates whether the category is archived

3. PRODUCTS

Stores product details, category association, price, stock, discount, description, image data, and product archive status.

Column	Data Type	Null	Key	Default	Description
id	INT	NO	PK, AUTO_INCREMENT	—	Unique product identifier
name	VARCHAR(180)	NO	—	—	Product name
category_id	INT	NO	FK	—	References CATEGORIES_id
price	DECIMAL(10,2)	NO	CHECK (>= 0)	—	Product price
stock	INT	NO	CHECK (>= 0)	0	Available product stock
discount	DECIMAL(5,2)	NO	CHECK (0–100)	0.00	Discount percentage
description	TEXT	YES	—	NULL	Product description
image	LONGBLOB	YES	—	NULL	Product image data
is_archived	TINYINT(1)	NO	—	0	Indicate whether the product is archived

4. ORDERS

Stores customer orders, order dates, delivery/contact details, order status, totals, and return/received-date information.

Column	Data Type	Null	Key	Default	Description
id	INT	NO	PK,AUTO_INCREMENT	AUTO_INCREMENT	Unique order identifier
customer_id	INT	NO	FK	None	References USERS.id for the customer account
order_date	DATETIME	NO	—	CURRENT_TIMESTAMP	Order date/time stored as text
location	VARCHAR(255)	YES	—	..	Delivery or pickup location; defaults to an empty string
contact	VARCHAR(80)	YES	—	..	Customer contact information; defaults to an empty string
status	VARCHAR(40)	NO	—	Pending	Order status; defaults to Pending
total	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Order total; must be zero or greater
return_reason	TEXT	YES	—	NULL	Return reason; defaults to an empty string
received_date	DATETIME	YES	—	NULL	Date the order was received; stored as text

5. ORDER_ITEMS

Associative (junction) table that stores the products included in each order, along with quantity and unit price.

Column	Data Type	Null	Key	Default	Description
id	INT	NO	PK, AUTO_INCREMENT	—	Unique order item identifier
order_id	INT	NO	FK	—	References ORDERS.id; the row is removed when its parent order is deleted
product_id	INT	NO	FK	—	References PRODUCTS.id
qty	INT	NO	CHECK (> 0)	—	Quantity ordered; must be greater than zero
unit_price	DECIMAL(10,2)	NO	CHECK (>= 0)	—	Unit price recorded for the order item; must be zero or greater

6. PAYMENTS

Stores payment details for orders. order_id is unique, so an order can have at most one payment record.

Column	Data Type	Null	Key	Default	Description
id	INT	NO	PK,AUTO_INCREMENT	—	Unique payment identifier
order_id	INT	NO	FK, UNIQUE	—	References ORDERS.id and prevents more than one payment row per order
method	VARCHAR(60)	NO	—	—	Payment method
status	VARCHAR(40)	NO	—	—	Payment status
reference	VARCHAR(190)	YES	—	..	Payment reference; defaults to an empty string
payment_date	DATETIME	YES	—	NULL	Payment date/time stored as text; defaults to an empty string

4. Foreign Key Integrity Matrix

Child Table	Foreign Key	Parent Table	Referenced PK	On Delete	On Update	Relationship
PRODUCTS	category_id	CATEGORIES	id	NO ACTION	NO ACTION	1:M (Optional)
ORDERS	customer_id	USERS	id	NO ACTION	NO ACTION	1:M (Optional)
ORDER_ITEMS	order_id	ORDERS	id	CASCADE	NO ACTION	1:M (Optional)
ORDER_ITEMS	product_id	PRODUCTS	id	NO ACTION	NO ACTION	1:M (Optional)
PAYMENTS	order_id	ORDERS	id	CASCADE	NO ACTION	1:0..1 (order may have no payment)

5. Data Definition Language (DDL) Script

The script below creates the six tables and four indexes used in the Cherry Cosmetics database.

```
CREATE DATABASE IF NOT EXISTS cherry_cosmetics
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE cherry_cosmetics;

CREATE TABLE IF NOT EXISTS users (
id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(100) NOT NULL UNIQUE,
email VARCHAR(190) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
role ENUM('Admin', 'Customer') NOT NULL,
is_archived TINYINT(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS categories (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL UNIQUE,
is_archived TINYINT(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS products (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(180) NOT NULL,
category_id INT NOT NULL,
price DECIMAL(10,2) NOT NULL,
stock INT NOT NULL DEFAULT 0,
discount DECIMAL(5,2) NOT NULL DEFAULT 0,
description TEXT,
image LONGBLOB,
is_archived TINYINT(1) NOT NULL DEFAULT 0,
INDEX idx_products_category (category_id),
CONSTRAINT fk_products_category
FOREIGN KEY (category_id)
REFERENCES categories(id)
ON DELETE NO ACTION
ON UPDATE NO ACTION
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS orders (
id INT AUTO_INCREMENT PRIMARY KEY,
customer_id INT NOT NULL,
order_date VARCHAR(40) NOT NULL,
location VARCHAR(255) NOT NULL DEFAULT '',
contact VARCHAR(80) NOT NULL DEFAULT '',
status VARCHAR(40) NOT NULL DEFAULT 'Pending',
total DECIMAL(12,2) NOT NULL DEFAULT 0,
return_reason TEXT,
received_date VARCHAR(40) NOT NULL DEFAULT '',
INDEX idx_orders_customer (customer_id),
CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_id)
REFERENCES users(id)
ON DELETE NO ACTION
ON UPDATE NO ACTION
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS order_items (
id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT NOT NULL,
product_id INT NOT NULL,
qty INT NOT NULL,
unit_price DECIMAL(10,2) NOT NULL,
INDEX idx_items_order (order_id),
INDEX idx_items_product (product_id),
CONSTRAINT fk_order_items_order
FOREIGN KEY (order_id)
REFERENCES orders(id)
ON DELETE CASCADE
ON UPDATE NO ACTION,
CONSTRAINT fk_order_items_product
FOREIGN KEY (product_id)
REFERENCES products(id)
ON DELETE NO ACTION
ON UPDATE NO ACTION
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS payments (
id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT NOT NULL UNIQUE,
method VARCHAR(60) NOT NULL,
status VARCHAR(40) NOT NULL,
reference VARCHAR(190) NOT NULL DEFAULT '',
payment_date VARCHAR(40) NOT NULL DEFAULT '',
CONSTRAINT fk_payments_order
FOREIGN KEY (order_id)
REFERENCES orders(id)
ON DELETE CASCADE
ON UPDATE NO ACTION
) ENGINE=InnoDB;
```